



Introduction to Neural Networks 2

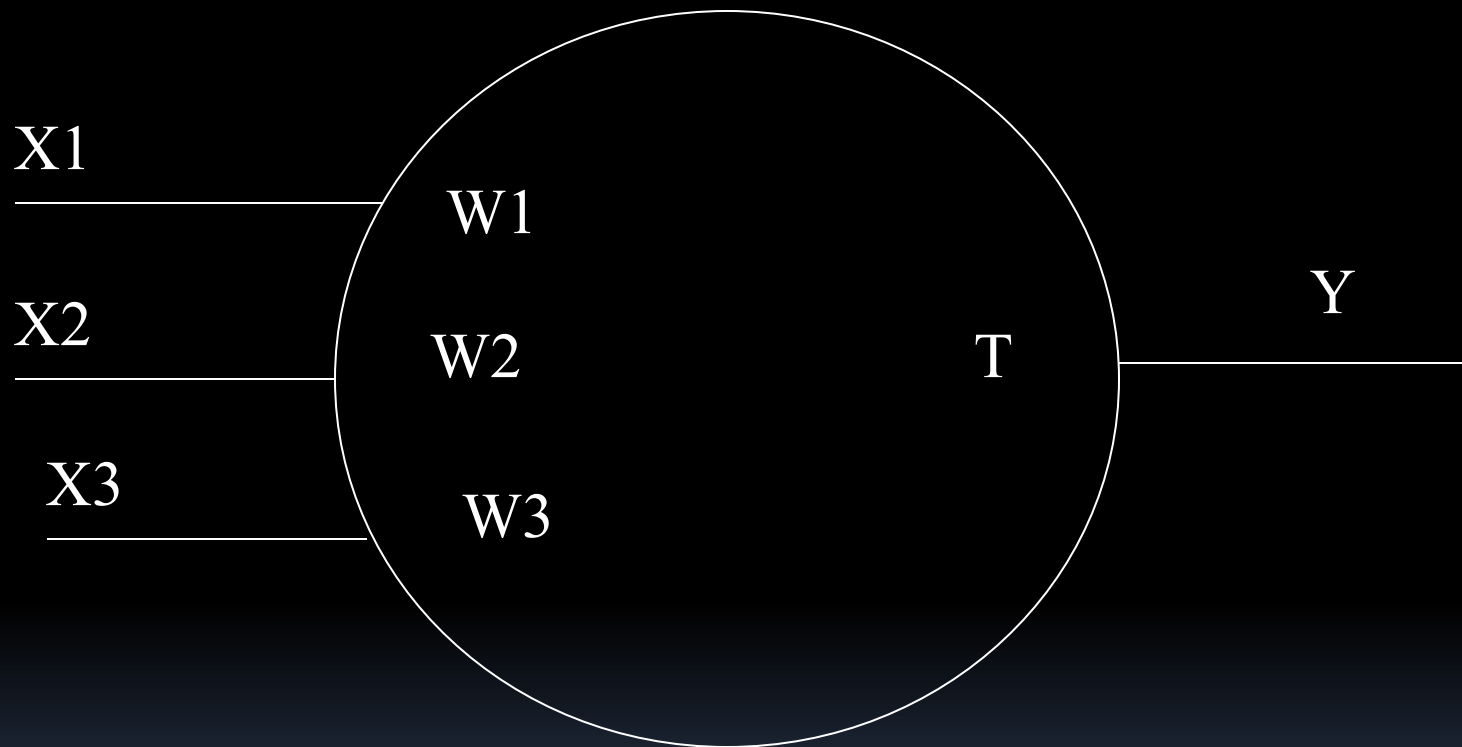
ARTIFICIAL INTELLIGENCE TECHNIQUES



Overview

- The McCulloch-Pitts neuron
 - Pattern space
 - Limitations
 - Learning
- 

McCulloch-Pitts model



$Y=1$ if $W_1X_1+W_2X_2+W_3X_3 \geq T$

$Y=0$ if $W_1X_1+W_2X_2+W_3X_3 < T$

Introduce the bias

Take the threshold over to the other side of the equation and replace it with a weight W_0 which equals $-T$, and include a constant input X_0 which equals 1.

$$Y=1 \text{ if } W_1X_1+W_2X_2+W_3X_3 - T \geq 0$$

$$Y=0 \text{ if } W_1X_1+W_2X_2+W_3X_3 - T < 0$$

$$Y=1 \text{ if } W_1X_1+W_2X_2+W_3X_3 + W_0X_0 \geq 0$$

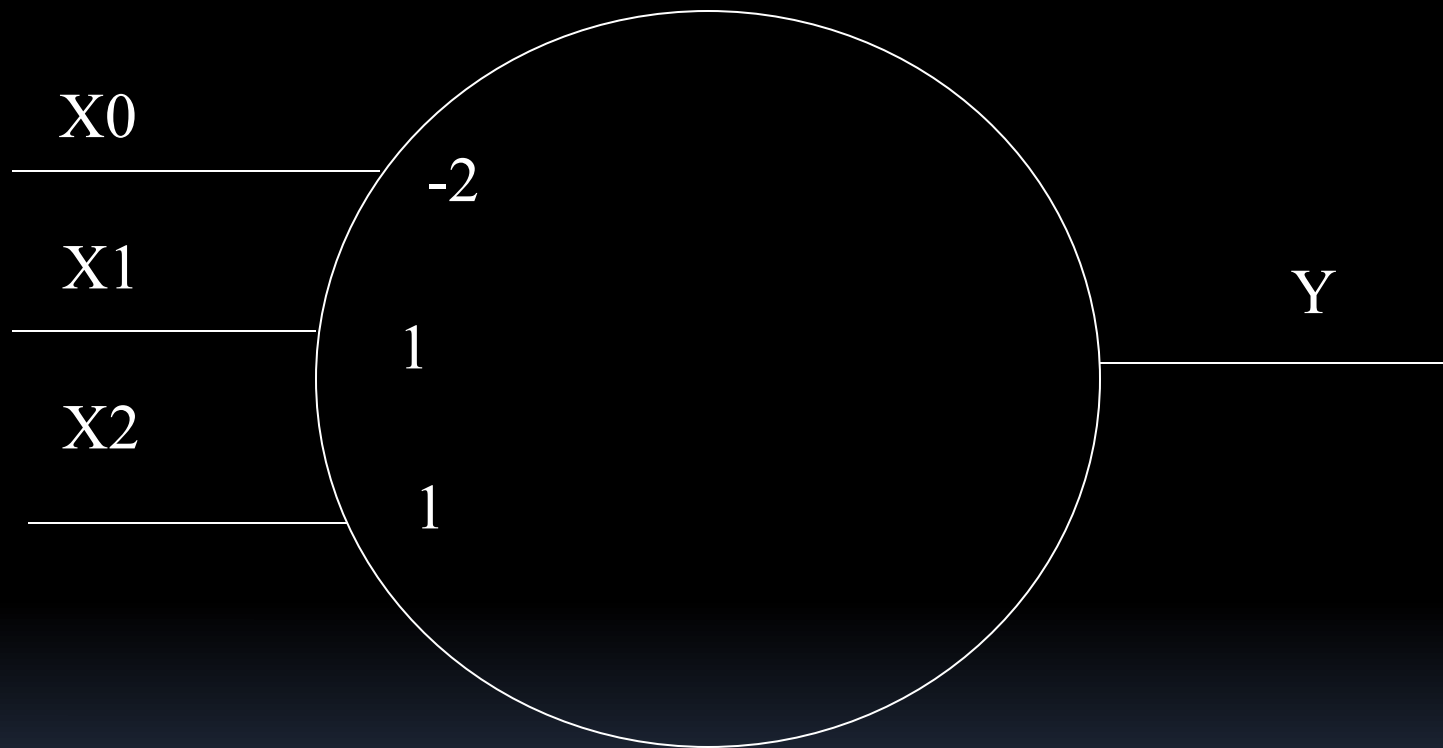
$$Y=0 \text{ if } W_1X_1+W_2X_2+W_3X_3 + W_0X_0 < 0$$

Logic functions - OR



$$Y = X1 \text{ OR } X2$$

Logic functions - AND



$$Y = X1 \text{ AND } X2$$

Logic functions - NOT



$$Y = \text{NOT } X1$$

The weighted sum

- The weighted sum, $\sum W_i X_i$ is called the “net” sum.
- $\text{Net} = \sum W_i X_i$
- $y=1$ if $\text{net} \geq 0$
- $y=0$ if $\text{net} < 0$

Hard-limiter

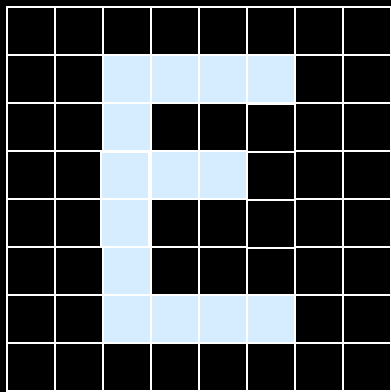
The threshold function is known as a hard-limiter.



When net is zero or positive, the output is 1, when net is negative the output is 0.

Example

Original image

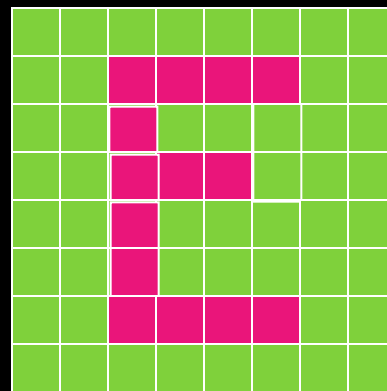


□ 0

■ +1

Net = 14

Weights

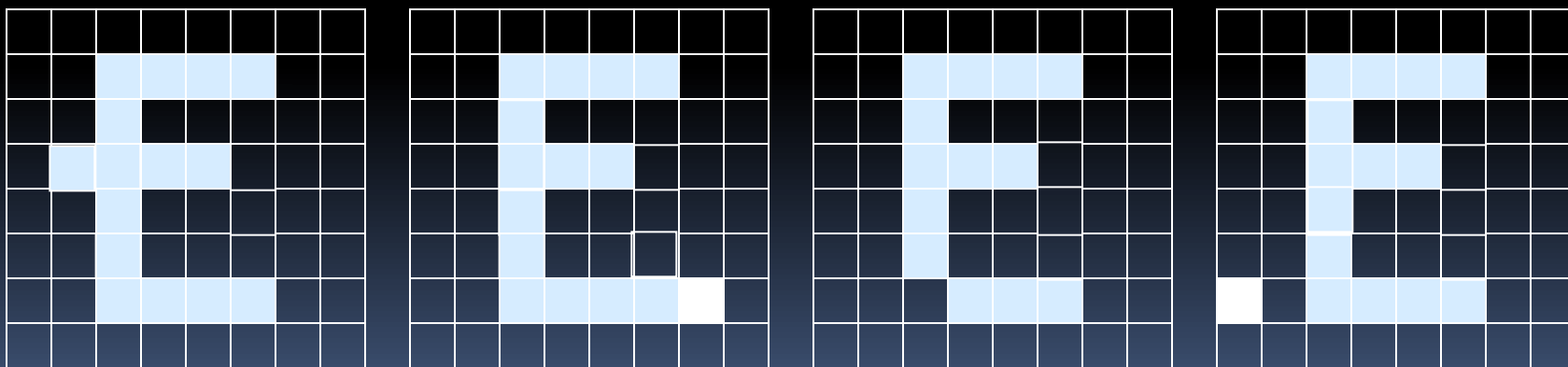


■ -1

■ +1

1000

Any pattern other than the original will produce a sum that is less than 0. If the bias is changed to -13, then patterns with 1 bit different from the original will give a sum that is 0 or more, so an output of 1.





Generalisation

- The neuron can respond to the original image and to small variations
 - The neuron is said to have generalised because it recognises patterns that it hasn't seen before
- 

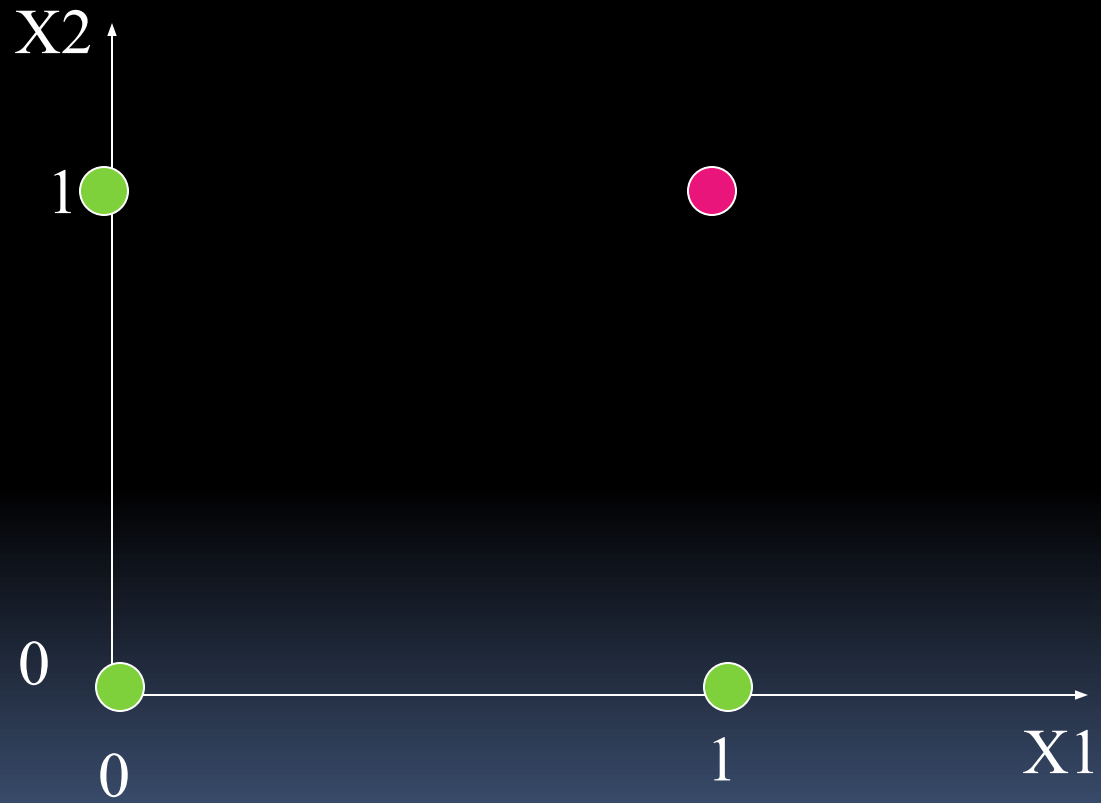


Pattern space

- To understand what a neuron is doing, the concept of pattern space has to be introduced
 - A pattern space is a way of visualizing the problem
 - It uses the input values as co-ordinates in a space
- 

Pattern space in 2 dimensions

X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



The AND function

● 1

● 0

Linear separability

The AND function shown earlier had weights of -2, 1 and 1. Substituting into the equation for net gives:

$$\text{net} = W_0X_0 + W_1X_1 + W_2X_2 = -2X_0 + X_1 + X_2$$

Also, since the bias, X_0 , always equals 1, the equation becomes:

$$\text{net} = -2 + X_1 + X_2$$

Linear separability

The change in the output from 0 to 1 occurs when:

$$\text{net} = -2 + X_1 + X_2 = 0$$

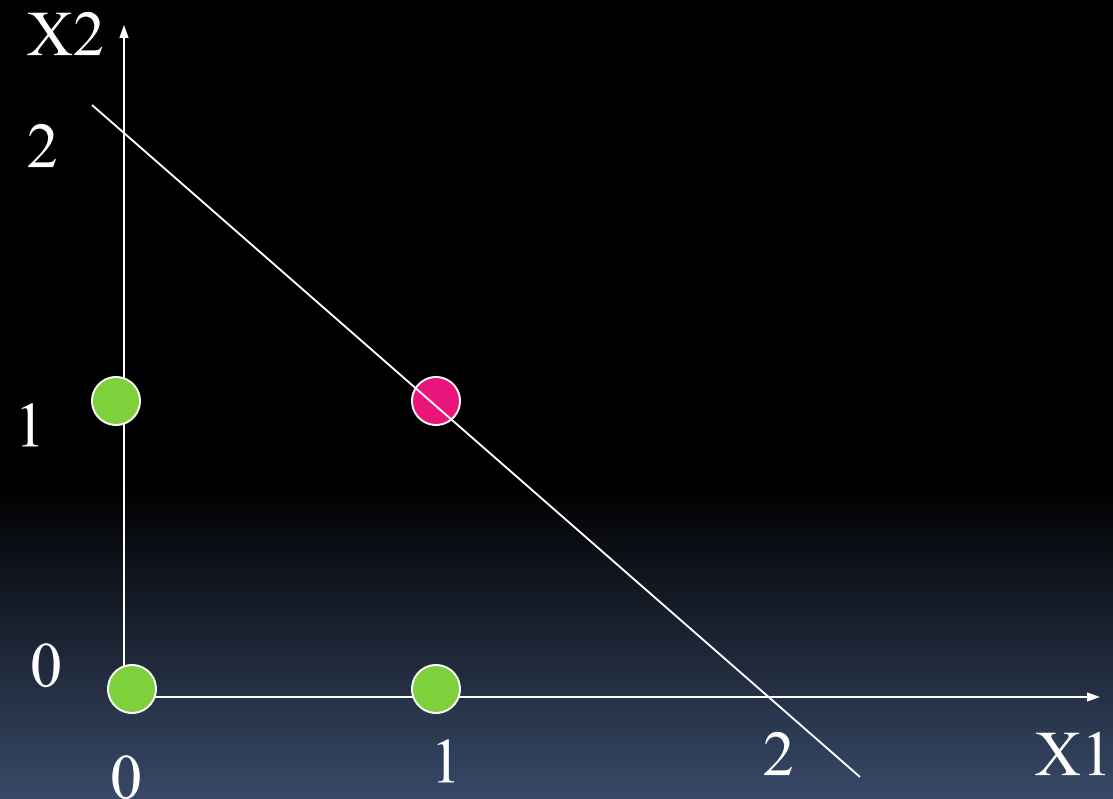
This is the equation for a straight line.

$$X_2 = -X_1 + 2$$

Which has a slope of -1 and intercepts the X_2 axis at 2.
This line is known as a decision surface.

Linear separability

X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



The AND function



Linear separability

- When a neuron learns it is positioning a line so that all points on or above the line give an output of 1 and all points below the line give an output of 0
 - When there are more than 2 inputs, the pattern space is multi-dimensional, and is divided by a multi-dimensional surface (or hyperplane) rather than a line
- 

- 
- 
- Demo
 - Demonstration on linearly/non-linearly separability
 - Image analysis
 - Generality demonstration

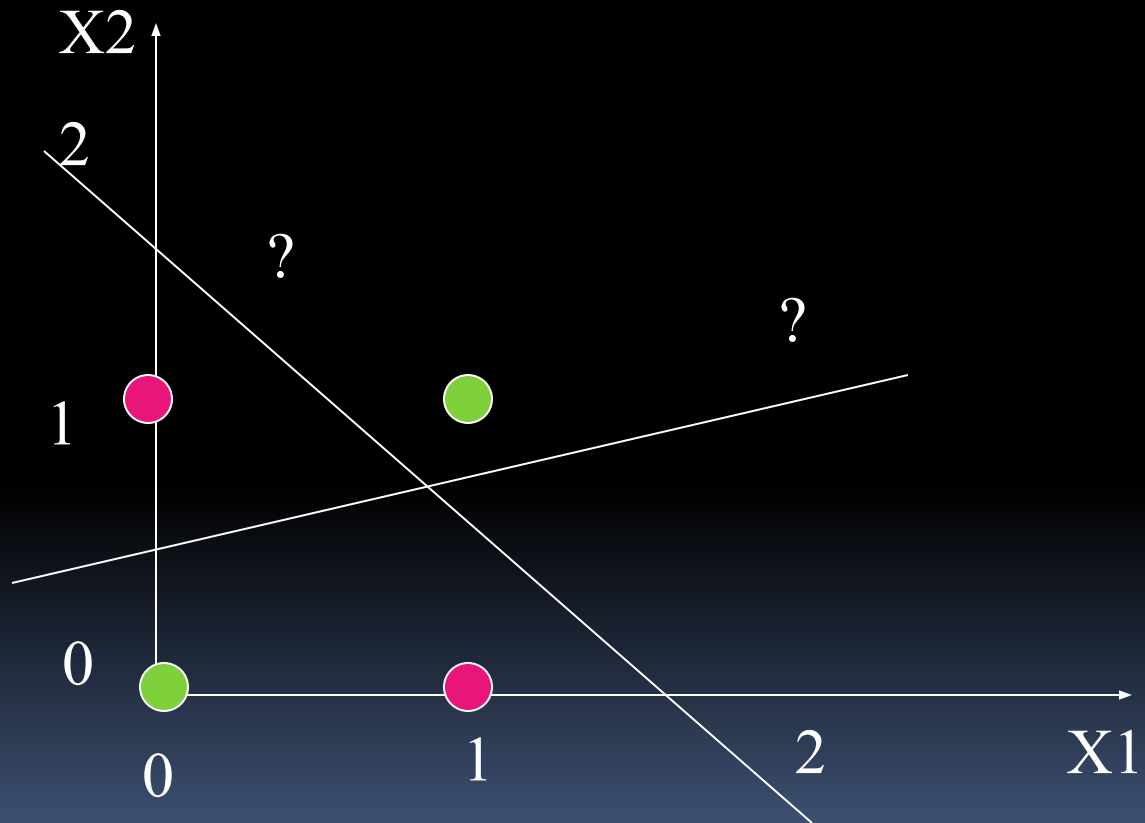


Are all problems linearly separable?

- No
 - For example, the XOR function is non-linearly separable
 - Non-linearly separable functions cannot be implemented on a single neuron
- 

Exclusive-OR (XOR)

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0





Learning

- A single neuron learns by adjusting the weights
 - The process is known as the delta rule
 - Weights are adjusted in order to minimise the error between the actual output of the neuron and the desired output
 - Training is supervised, which means that the desired output is known
- 

Delta rule

The equation for the delta rule is:

$$\Delta W_i = \eta X_i \delta = \eta X_i (d - y)$$

where d is the desired output and y is the actual output.
The Greek “eta”, η , is a constant called the learning coefficient
and is usually less than 1.

ΔW_i means the change to the weight, W_i .

Delta rule

- The change to a weight is proportional to X_i and to $d - y$.
- If the desired output is bigger than the actual output then $d - y$ is positive
- If the desired output is smaller than the actual output then $d - y$ is negative
- If the actual output equals the desired output the change is zero

Changes to the weight

	Output less than desired	Output more than desired
X_i is positive	Change is positive	Change is negative
X_i is negative	Change is negative	Change is positive



Example

- Assume that the weights are initially random
 - The desired function is the AND function
 - The inputs are shown one pattern at a time and the weights adjusted
- 

The AND function

X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Example

Start with random weights of 0.5, -1, 1.5

When shown the input pattern 1 0 0 the weighted sum is:

$$\text{net} = 0.5 \times 1 + (-1) \times 0 + 1.5 \times 0 = 0.5$$

This goes through the hard-limiter to give an output of 1.
The desired output is 0. So the changes to the weights are:

W0 negative

W1 zero

W2 zero

Handwritten notes:

$$\Delta w = \eta (d - a) a$$
$$w_{\text{new}} = w_{\text{old}} + \Delta w$$
$$\eta = 0.1$$

Handwritten calculations for weight updates:

$$w_0 = 0.5 - 0.1 = 0.4$$
$$w_1 = 0.5 - 0.1 = 0.4$$
$$w_2 = 0.5 - 0.1 = 0.4$$

Example

New value of weights (with η equal to 0.1) of 0.4, -1, 1.5

When shown the input pattern 1 0 1 the weighted sum is:

$$\text{net} = 1 \times 0.4 + (-1) \times 0 + 1.5 \times 1 = 1.9$$

This goes through the hard-limiter to give an output of 1.

The desired output is 0. So the changes to the weights are:

W0 negative

W1 zero

W2 negative

Example

New value of weights of 0.3, -1, 1.4

When shown the input pattern 1 1 0 the weighted sum is:

$$\text{net} = 1 \times 0.3 + (-1) \times 1 + 1.4 \times 0 = -0.7$$

Handwritten calculation showing the weighted sum result:

$$= 0$$
$$= -0.7$$

This goes through the hard-limiter to give an output of 0.
The desired output is 0. So the changes to the weights are:

W0zero

W1zero

W2zero

Example

New value of weights of 0.3, -1, 1.4

When shown the input pattern 1 1 1 the weighted sum is:

$$\text{net} = 1 \times 0.3 + (-1) \times 1 + 1.4 \times 1 = 0.7$$

This goes through the hard-limiter to give an output of 1.

The desired output is 1. So the changes to the weights are:

W_0 zero

W_1 zero

W_2 zero

Example - with $\eta = 0.5$

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	0.5	-1.0	1.5	0.5	1	-0.5
1	0	1	0.0	-1.0	1.5	1.5	1	-0.5
1	1	0	-0.5	-1.0	1.0	-1.5	0	0
1	1	1	-0.5	-1.0	1.0	-0.5	0	0.5

Example

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	0.0	-0.5	1.5	0.0	1	-0.5
1	0	1	-0.5	X0	1.5	1.0	1	-0.5
1	1	0	-1.0	-0.5	1.0	-1.5	0	0
1	1	1	-1.0	-0.5	1.0	-0.5	0	0.5

Example

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	-0.5	0.0	1.5	-0.5	0	0
1	0	1	-0.5	0.0	1.5	1.0	1	-0.5
1	1	0	-1.0	0.0	1.0	-1.0	0	0
1	1	1	-1.0	0.0	1.0	0.0	1	0

Example

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	-1.0	0.0	1.0	-1.0	0	0
1	0	1	-1.0	0.0	1.0	0.0	1	-0.5
1	1	0	-1.5	0.0	0.5	-1.5	0	0
1	1	1	-1.5	0.0	0.5	-1.0	0	0.5

Example

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	-1.0	0.5	1.0	-1.0	0	0
1	0	1	-1.0	0.5	1.0	0.0	1	-0.5
1	1	0	-1.5	0.5	0.5	-1.0	0	0
1	1	1	-1.5	0.5	0.5	-0.5	0	0.5

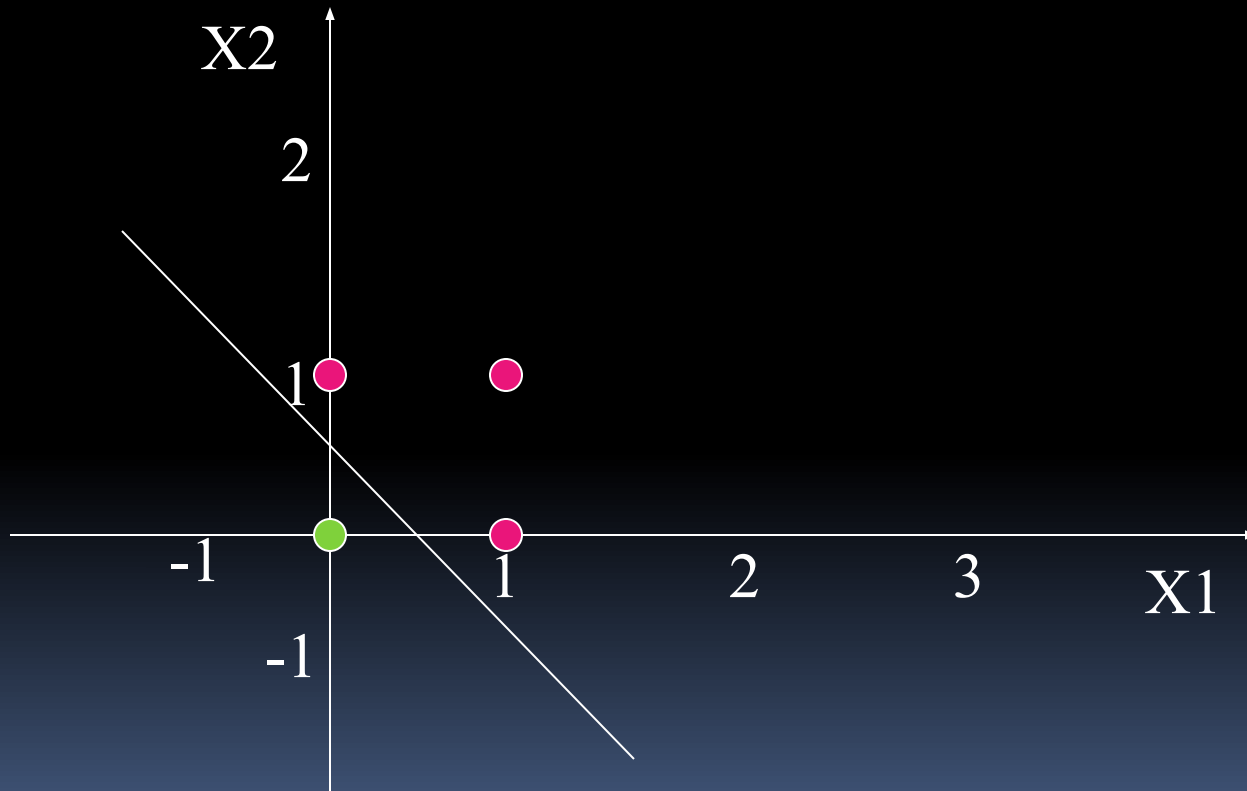
Example

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	-1.0	1.0	1.0	-1.0	0	0
1	0	1	-1.0	1.0	1.0	0.0	1	-0.5
1	1	0	-1.5	1.0	0.5	-0.5	0	0
1	1	1	-1.5	1.0	0.5	0.0	1	0

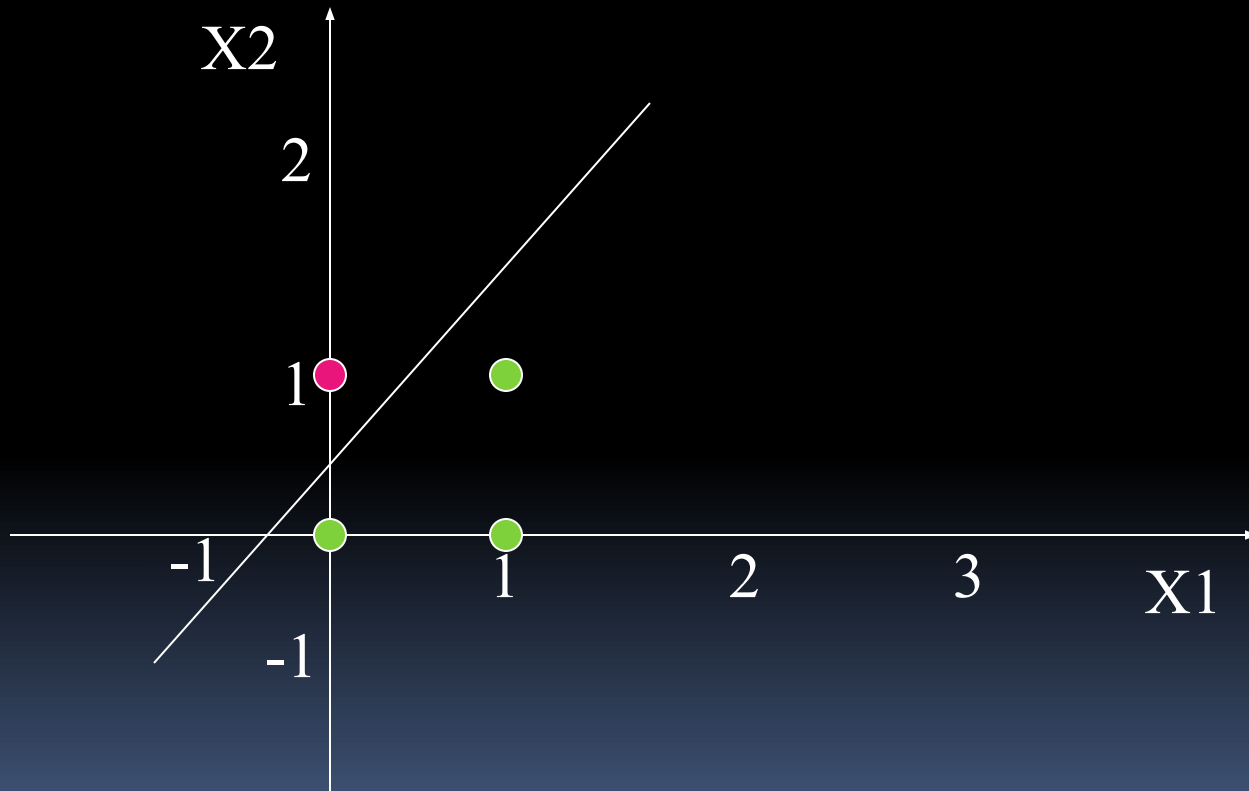
Example

X0	X1	X2	W0	W1	W2	Net	Y	0.5δ
1	0	0	-1.5	1.0	0.5	-1.5	0	0
1	0	1	-1.5	1.0	0.5	-1.0	0	0
1	1	0	-1.5	1.0	0.5	-0.5	0	0
1	1	1	-1.5	1.0	0.5	0.0	1	0

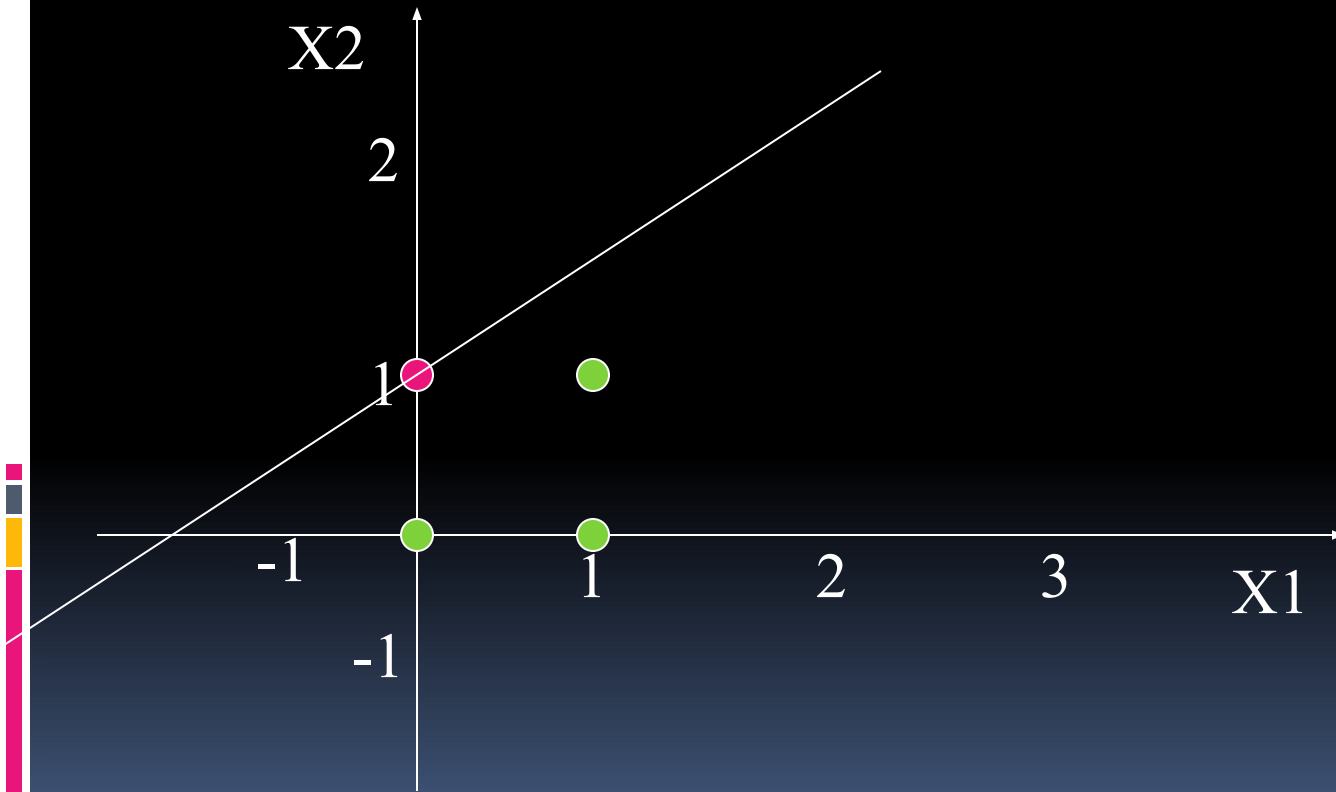
What happened in pattern space



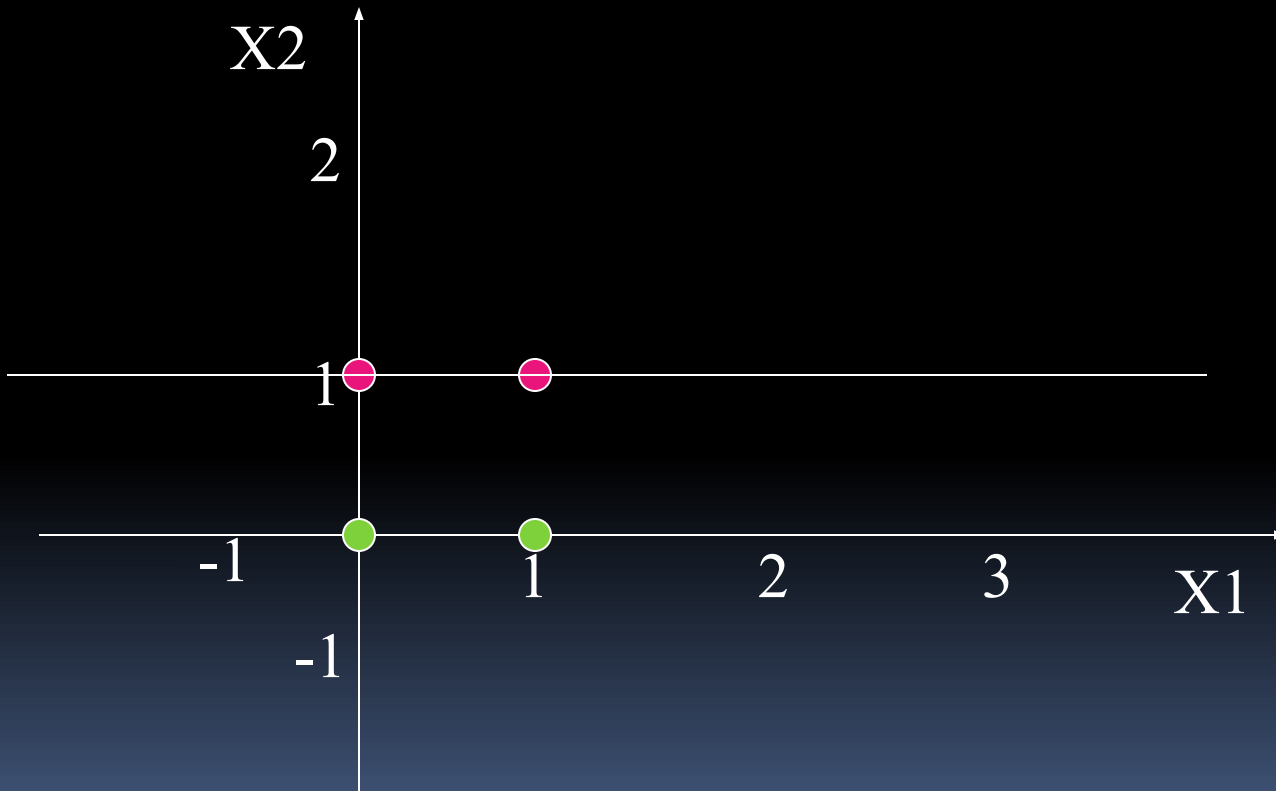
What happened in pattern space



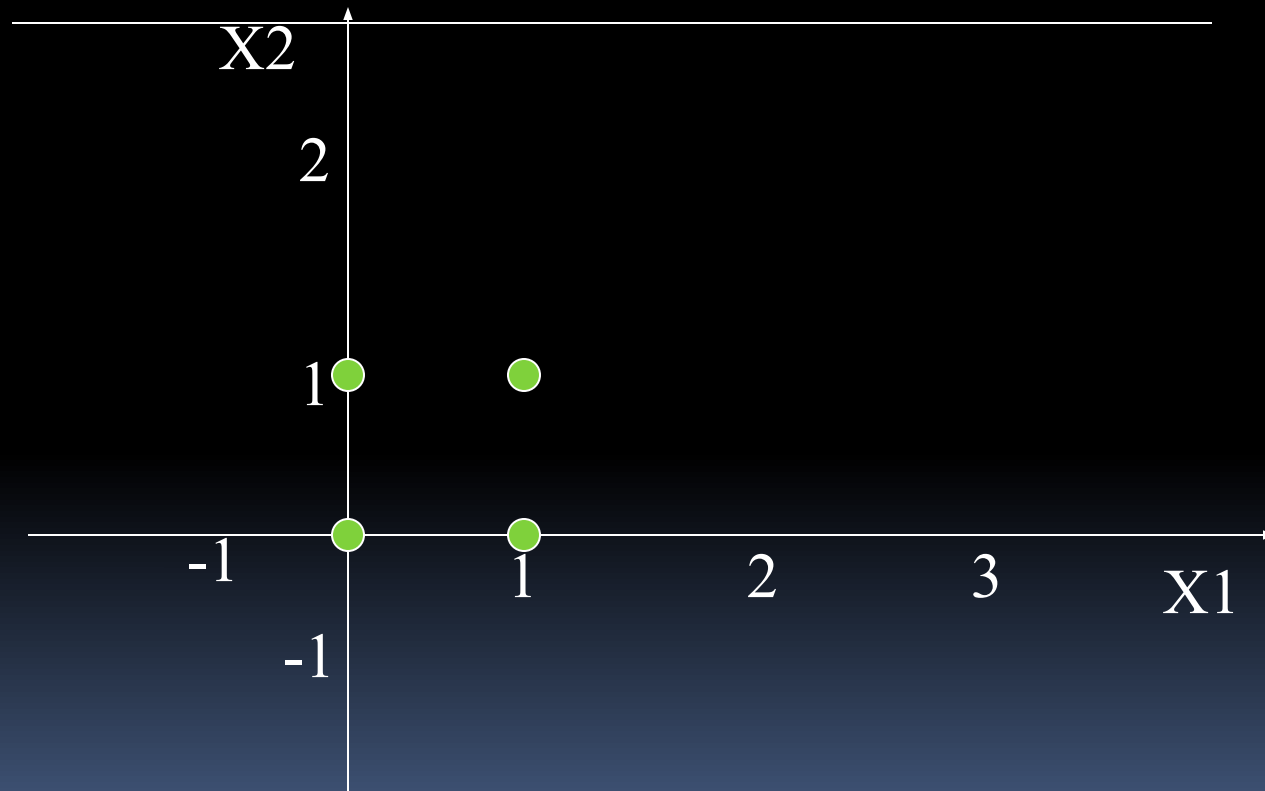
What happened in pattern space



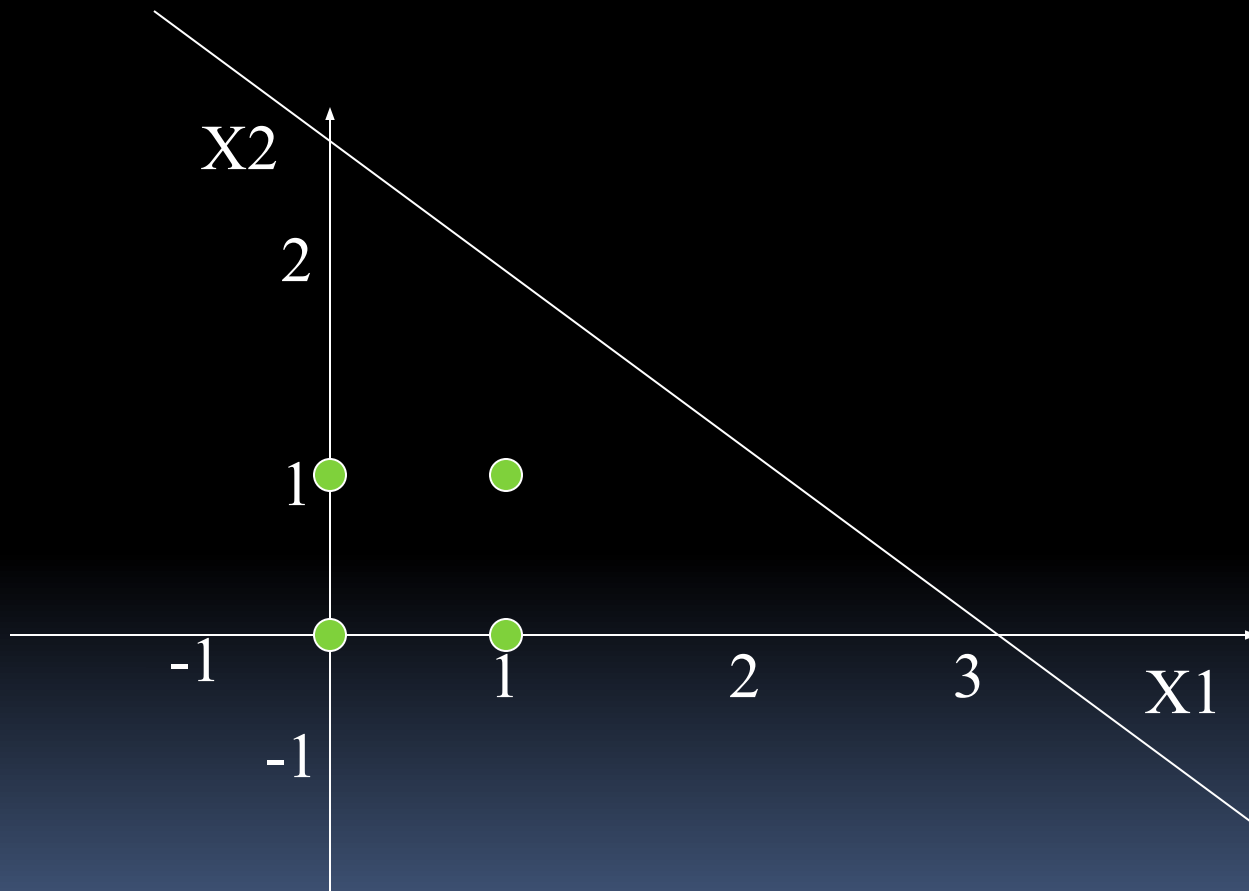
What happened in pattern space



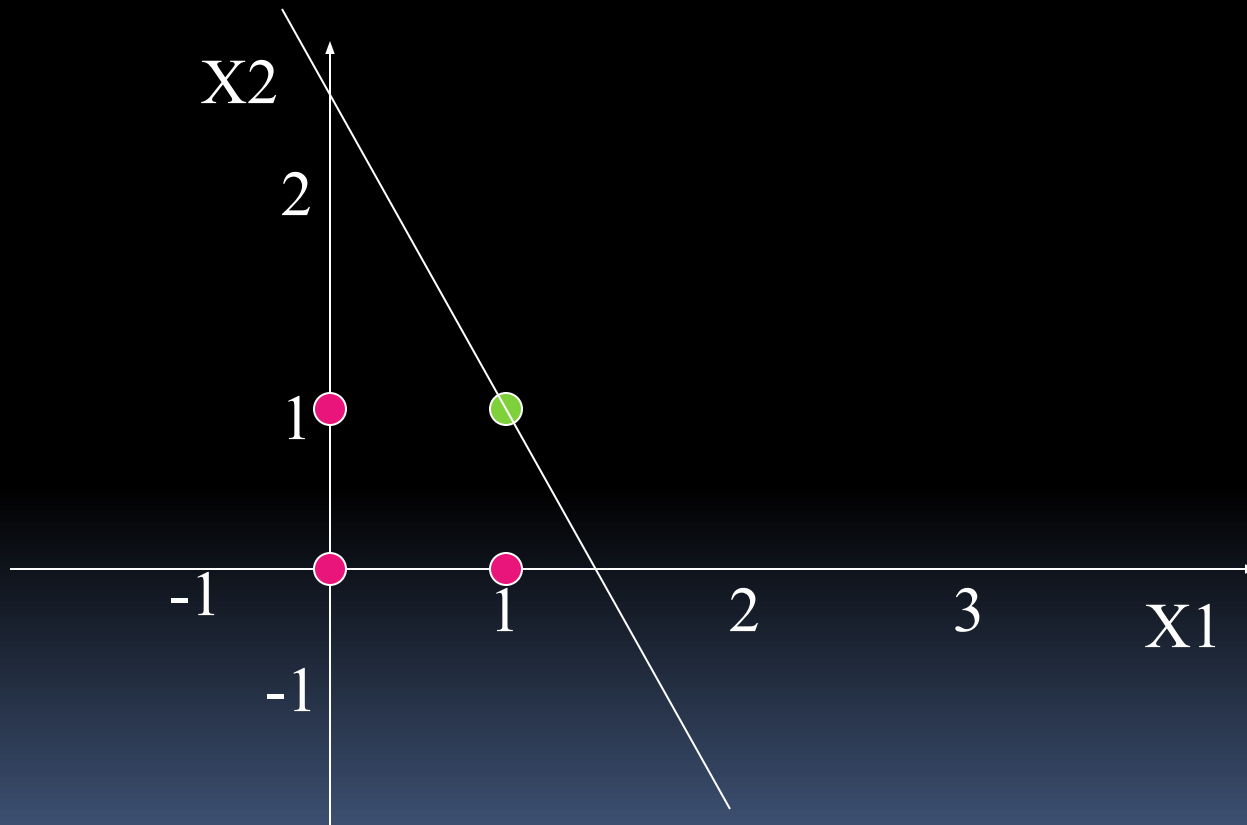
What happened in pattern space



What happened in pattern space



What happened in pattern space





Conclusions

- A single neuron can be trained to implement any linearly separable function
 - Training is achieved using the delta rule which adjusts the weights to reduce the error
 - Training stops when there is no error
 - Training is supervised
- 



Conclusions

- To understand what a neuron is doing, it helps to picture what's going on in pattern space
 - A linearly separable function can divide the pattern space into two areas using a hyperplane
 - If a function is not linearly separable, networks of neurons are needed
- 